

ΠΡΟΓΡΑΜΜΑΤΙΣΤΙΚΗ ΑΣΚΗΣΗ

Μεταγλωττιστής για Greek_plus_plus

Μάθημα: Μεταφραστές PLY602

ΜΠΟΥΖΟΥΚΑΣ ΚΩΝΣΤΑΝΤΙΝΟΣ-ΛΕΥΤΕΡΗΣ 5302

ΝΙΚΟΛΑΚΟΠΟΥΛΟΣ ΑΡΙΣΤΕΙΔΗΣ 5308

Ημερομηνία: 21/5/2025

Φάση Πρώτη:

Λεκτική Και Συντακτική Ανάλυση

Για τον Λεκτικό αναλυτή υλοποιήσαμε μια συνάρτηση «`get_token()`» η οποία λαμβάνει κάθε σύμβολο του αρχείου κώδικα και αναλόγως με την κατηγορία στην οποία ανήκει το κατηγοριοποιεί, η `get_token()` λειτουργεί κάθε φορά που την καλεί ο συντακτικός αναλυτής. Μέσα σε όλες τις συναρτήσεις υπάρχει μια `global variable` .

Για τον συντακτικό αναλυτή ορίσαμε μια συνάρτηση η οποία λαμβάνει τα σύμβολα και υλοποιεί την ανάλυση για κάθε μπλόκ. Δηλαδή περιμένει κάποια συγκεκριμένα `tokens` στην αρχή , αργότερα καλεί την `block` η οποία είναι υπεύθυνη για την συντακτική ανάλυση του `sub block` κα του `main_block`.

Συγκεκριμένα η `block` λαμβάνει `tokens` τα οποία περιμένει να είναι οτιδήποτε μπορεί κανείς να βρει στον κώδικα ενός προγράμματος και ολοκληρώνεται όταν ο λεκτικός λάβει κάποια από τις τερματικές λέξεις (`τέλος_εάν`, `τέλος_συνάρτησης` κλπ). Ανάλογα με την λέξη κλειδί που θα βρει καλεί μία συνάρτηση για την εκάστοτε περίπτωση, ενδεικτικά αν βρει το `token` “`εάν`” καλεί την “`if_stat()`” αν βρεί `token` που ανήκει στην οικογένεια των `id` κάλει την “`assign()`” καθώς δεν μπορεί να υπάρχει `id` στο πρόγραμμα δίχως κάποια άλλη λέξη κλειδί.

Οι επιπρόσθετες συναρτήσεις για την λειτουργία

συγκεκριμένων block ελέγχουν την σύνταξη του κώδικα που περιμένει ο συντακτικός αν δει μία λέξη κλειδί, στο παράδειγμα με την `if_stat()` αφού έχει λάβει το “εάν” αμέσως μετά περιμένει να βρει ένα condition το οποίο το αναλύουμε με ένα από τα επιπλέον εργαλεία μας τα οποία χρησιμοποιούμε μέσα στις συναρτήσεις για βοήθεια στην ανάλυση. Συνεχίζοντας το παράδειγμα με το εάν , αφού έχει λάβει το condition περιμένει να βρει την λέξη “τότε” και αργότερα καλεί την block, αφού τερματιστεί η λειτουργία της η συνάρτηση μας ψάχνει να βρει την λέξη “αλλιώς” ή την “εάν_τέλος” όπου με την δεύτερη θα τερματίσει την λειτουργία της (η `if_stat`) και με την πρώτη θα ξανά καλέσει την block.

Τα εργαλεία μας, που δημιουργήσαμε για βοήθεια περιλαμβάνουν τις συναρτήσεις `condition`, `expression`, `boolterm` οι οποίες καλούνται όποτε περιμένουμε από τον κώδικα συγκεκριμένα κάτι από αυτά.

Συνοψίζοντας, ο συντακτικός αναλυτής λειτουργεί λαμβάνοντας tokens από τον λεκτικό και περιμένει συγκεκριμένα μοτίβα από τον κώδικα. Αν τα λαμβάνοντα σύμβολα συμβαδίζουν με όσα περιμένει ο αναλυτής τότε ολοκληρώνεται και η συντακτική ανάλυση επιτυχώς.
Ακολουθούν κομμάτια κώδικα για την επίδειξη των προαναφερθέντων.

```

def get_token():
    global tok
    global ind
    global f_str
    global line_ind
    ret = ""

    while True:
        tem = 0
        if ind >= len(f_str) - 1:
            return "eof"
        if f_str[ind] == " " or f_str[ind] == "\t" or f_str[ind] == "":
            ind += 1
            pass
        elif f_str[ind] == "\n":
            line_ind += 1
            ind += 1
            pass
        else:
            ret = ret + f_str[ind]
            ind += 1
            break

```

#Αναγνωριστικό

```

if ret.isalpha():
    tem = 1
    f = "id"
    while f_str[ind].isalnum() or f_str[ind] == "_":
        ret = ret + f_str[ind]
        if ind + 1 >= len(f_str):
            break
        ind += 1

```

#Αριθμός

```

if ret.isnumeric():
    tem = 1
    f = "number"
    while True:
        if f_str[ind].isnumeric():
            ret = ret + f_str[ind]
            ind += 1
        elif f_str[ind].isalpha():
            ret = error(1)
            return ret
        else:
            break

```

#delimiters

```

if ret == "," or ret == ";":
    tem = 1
    f = "delimiter"

```

```
#Comments
if ret == "{" or ret == "}":
    tem = 1
    f = "Comment_Operator"

#Relational
if ret == "<" or ret == ">" or ret == "=" :
    tem = 1
    f = "rel_Operator"
    if ret == "<" and f_str[ind] == "=":
        ret = ret + f_str[ind]
        ind +=1
    if ret == "<" and f_str[ind] == ">":
        ret = ret + f_str[ind]
        ind +=1
    if ret == ">" and f_str[ind] == "=":
        ret = ret + f_str[ind]
        ind +=1

#:
if ret == ":":
    tem = 1
    if f_str[ind] == "=":
        ret = ret + f_str[ind]
        f = "AssignOperator"
        ind+=1
    else:
        f = "delimiter"
```

```

#math Operators
if ret == "-" and tok.family!= "number" and tok.family != "id":
    tem = 1

    if f_str[ind].isnumeric():
        f = "number"
        while True:
            if f_str[ind].isnumeric():
                ret = ret + f_str[ind]
                ind +=1
            elif f_str[ind].isalpha():
                ret = error(1)
                return ret
            else:
                break
    elif f_str[ind].isalpha():
        f = "id"
        while f_str[ind].isalnum() or f_str[ind] == "_":
            ret = ret + f_str[ind]
            if ind + 1 >= len(f_str):
                break
            ind+=1
    else:
        f = "Math_Operator"
elif ret == "+" or ret == "-" or ret == "*" or ret == "/" :
    tem = 1
    f = "Math_Operator"

```

```

#%
if ret == "%":
    tem = 1
    f = "%_operator"

if tem == 0 and isComm == False:
    tokk = error(4)
else:
    tokk = token(ret,f,line_ind)

if tokk == "{" and in_int == 1:
    while tokk.str != "}":
        tokk = get_token()
return tokk

```

```

def block():
    global tok
    global line_ind
    global isComm
    global can_command
    prev_tok = tok
    if type(tok) == error:
        pass
    else:
        while True:
            if type(tok) == error:
                break
            else:
                if tok.str == "αρχή_προγράμματος":
                    can_command = True
                if tok.str == ";":
                    tok = get_token()
                    break
                if tok == "eof":
                    tok = prev_tok
                    break
                elif tok.str in end_keywords:
                    break
                elif tok.str == "αλλιώς":
                    break
                elif tok.str == "δήλωση" :
                    state()
                elif tok.str == "δηλώσεις":
                    states()
                elif tok.str == "εάν":
                    if_stat()
                elif tok.str == "επανάλαβε":
                    rep_unt()
                elif tok.str == "όσο":
                    while_stat()
                elif tok.str == "για":
                    for_stat()

```

Φάση Δεύτερη:

Ενδιάμεσος κώδικας και Πίνακας Συμβόλων

Για τον Ενδιάμεσο Κώδικα αρχικά υλοποιήσαμε τις βοηθητικές συναρτήσεις που δώθηκαν από τις διαφάνειες (`gen_quad()`, `backpatch()` κλπ). Αργότερα μέσα στην συνάρτηση `produce_int_code()` ακολουθήσαμε την ροή της συντακτικής ανάλυσης, δηλαδή διάβασμα των `token` και παραγωγή των τετράδων και αποθήκευση με `label` για κάθε μοτίβο που υπάρχει στην γραμματική.

Οι διαφορές με την συντακτική ανάλυση είναι ότι υπάρχει ξεχωριστή συνάρτηση που υλοποιεί το `subblock` και ξεχωριστή για το `main block`. Συγκεκριμένα η `sub` λειτουργεί για τις δηλώσεις και τις συναρτήσεις και τρέχει έως ότου να βρει “αρχή προγράμματος” αλλά επίσης και στην δημιουργία του πίνακα συμβόλων.

Η `interblock`, η οποία προφανώς καλείται και για τα εσωτερικά `block` των συναρτήσεων, κάνει την δουλειά της παραγωγής του κώδικα (παρόμοια με της `block()`), δηλαδή για κάθε σύμβολο λαμβάνει το `pattern`, που έχει ήδη ελεγχθεί από τον συντακτικό, και με την βοήθεια των εργαλείων και συναρτήσεων για τις λέξεις κλειδιά παράγει τον ενδιάμεσο κώδικα.

Για τον πίνακα συμβόλων, αυτό που κάναμε ήταν, να δημιουργίσουμε κάποιες δομές, `scope`, `entity` κλπ όπου κρατάν πληροφορίες για τον πίνακα. Επιπλέον μια πολύ σημαντική δομή είναι η λίστα η οποία για κάθε `scope` κρατάει την “απόσταση” από την στοίβα.

Συγκεκριμένα η λειτουργία της παραγωγής του πίνακα είναι, κάθε φορά που δηλώνεται μια μεταβλητή ,είτε με την λέξη κλειδί “Δήλωση” είτε μέσα σε μία συνάρτηση με “είσοδος”, “εξοδος” είτε στον ορισμός μίας συνάρτησης είτε στην δημιουργία temporary μεταβλητών για την χρήση τους στον ενδιάμεσο κώδικα, ένα αντικείμενο entity παράγεται και αποθηκεύεται μέσα στον πίνακα από scopes στο scope του block που βρισκόμαστε, και παίρνει τις ιδιότητες που του αναλογούν,(μεταβλητή,temp μεταβλητή,συνάρτηση,παράμετρος).

Στην παραγωγή του τελικού θα αναφέρω πως τον χρησιμοποιήσαμε.

Ακολουθούν κομμάτια κώδικα για την επίδειξη των προαναφερθέντων.

```
#Int Code Producer
def produce_int_code():
    mkfile = open("int_Code.int","w")
    global tok
    global pr_name
    scope([],0)
    tok = get_token()
    if tok.str == "πρόγραμμα":
        tok = get_token()
        pr_name = tok.str

    subblock()
    gen_quad("begin_block",pr_name,"_","_")
    interblock()
    if tok.str == "τέλος_προγράμματος":
        gen_quad("halt","_","_","_")
        gen_quad("end_block","_","_","_")
    for i in quads:
        mkfile.write(str(i)+"\n")
```

```

#BLOCKS
def subblock():
    global curr_block
    global tok
    global interfunc_list
    global curr_scope
    global offsets
    global scopes
    global is_in_f
    fType = 0
    while tok.str != "αρχή_προγράμματος":
        if tok.str == "δήλωση":
            tok = get_token()
            while tok.str not in keywords:
                if tok.family == "id":
                    scopes[curr_scope].ent_list.append(entity([0,tok.str,offsets[curr_scope]]))
                    offsets[curr_scope] += 4
                    tok = get_token()
                else:
                    tok = get_token()
            elif tok.str == "συνάρτηση" or tok.str == "διαδικασία":
                is_in_f = True
                if tok.str == "διαδικασία":
                    fType = 1
                tok = get_token()
                interfunc_list.append(interfunc(tok.str,fType))
                gen_quad("begin_block",tok.str,"_", "_")
                ql = next_quad()
                scopes[curr_scope].ent_list.append(entity([1,tok.str,fType,ql,[]]))
                curr_block = tok.str
                offsets[curr_scope] +=4
                tok = get_token()
                count_args = 0
                if tok.str == "(":
                    while tok.str != ")":
                        tok = get_token()
                        if tok.family == "id":
                            count_args +=1
                    tok = get_token()

```

```

scopes[curr_scope] += 1
1384         while count_args > 0:
1385             if tok.str == "είσοδος":
1386                 tok = get_token()
1387                 while tok.str not in keywords:
1388                     if tok.family == "id":
1389                         temp = entity([2,tok.str,"CV",offsets[curr_scope]])
1390                         for i in scopes[curr_scope - 1].ent_list:
1391                             if i.name == curr_block:
1392                                 i.args.append(temp)
1393                         scopes[curr_scope].ent_list.append(temp)
1394                         offsets[curr_scope] +=4
1395                         count_args -=1
1396                         tok = get_token()
1397             elif tok.str == "έξοδος":
1398                 tok = get_token()
1399                 if tok.family == "id":
1400                     temp = entity([2,tok.str,"REF",offsets[curr_scope]])
1401                     for i in scopes[curr_scope - 1].ent_list:
1402                         if i.name == curr_block:
1403                             i.args.append(temp)
1404                     scopes[curr_scope].ent_list.append(temp)
1405                     offsets[curr_scope] +=4
1406                     count_args -=1
1407                     tok = get_token()
1408             else:
1409                 tok = get_token()
1410             subblock()
1411
1412         for i in scopes[curr_scope - 1].ent_list:
1413             if curr_block == i.name:
1414                 i.framelength = offsets[curr_scope]
1415         if quads[len(quads) - 1].op == "end_block":
1416             produce_final_code()
1417         del scopes.append(scopes[curr_scope])
1418         scopes.pop(curr_scope)
1419         offsets.pop(curr_scope)
1420         curr_scope -=1
1421         is_in_f = False

```

```

def interblock():
    global tok
    global curr_block
    end_keywords.append("αλλιώς")
    while tok.str not in end_keywords:

        if tok.family == "id":
            tt = 0
            if tok.str in funclist:
                tt = 1

            temp = tok.str
            tok = get_token()
            if tok.str == "!=":
                temp2 = inter_assign()
                gen_quad("!=" , temp2, "_", temp)
            if tt == 1:
                gen_quad("retv", temp, "_", "_")

        elif tok.str == "διάβασε":
            tok = get_token()
            inter_read()
            tok = get_token()
        elif tok.str == "γράψε":
            tok = get_token()
            inter_write()
        elif tok.str == "εκτέλεσε":
            tok = get_token()
            inter_exec(tok.str)
        elif tok.str == "εάν":
            tok = get_token()
            inter_if()
            if tok.str != "τέλος_προγράμματος":
                tok = get_token()
        elif tok.str == "επανάλαβε":
            tok = get_token()
            inter_repeat()
            if tok.str != "τέλος_προγράμματος":
                tok = get_token()

```

```

✓ def gen_quad(op,op1,op2,op3):
    q = quad(op,op1,op2,op3)
    quads.append(q)

    return q

✓ def next_quad():
    global label_ind
    return label_ind

✓ def new_temp():
    global temp_ind
    t = f'T_{temp_ind}'
    temp_ind+=1

    scopes[curr_scope].ent_list.append(entity([3,t,offsets[curr_scope]]))
    offsets[curr_scope] +=4
    return t

> def empty_list(): ...

> def make_list(label): ...

> def merge_list(l1,l2): ...

✓ def backpatch(l,label):
    global quads
    for i in l:
        for j in quads:
            if i == j.label:
                j.op3 = label

```

```

1626 #Record Structures and globals
1627 scopes = []
1628 offsets = []
1629 curr_scope = 0
1630 is_in_f = False
1631 del_scopes = []
1632 class entity:
1633     def __init__(self,args):
1634         if args[0] == 0: #Variable
1635             self.ent_type = 0
1636             self.name = args[1]
1637             self.offset = args[2]
1638         if args[0] == 1: #function
1639             self.ent_type = 1
1640             self.name = args[1]
1641             self.type = args[2]
1642             self.start_quad = args[3]
1643             self.args = args[4]
1644             self.framelength = 0
1645         if args[0] == 2: #args
1646             self.ent_type = 2
1647             self.name = args[1]
1648             self.mode = args[2]
1649             self.offset = args[3]
1650         if args[0] == 3: #temp
1651             self.ent_type = 0
1652             self.name = args[1]
1653             self.offset = args[2]
1654
1655 class scope:
1656     def __init__(self,ent_list,nest_ind):
1657         global curr_scope
1658         global offsets
1659         global scopes
1660         self.ent_list = ent_list
1661         self.nest_ind = nest_ind
1662         scopes.append(self)
1663         offsets.append(12)
1664         curr_scope = nest_ind

```

Φάση Τρίτη:

Παραγωγή Τελικού Κώδικα

Ολοκληρώνοντας, για την υλοποίηση του τελικού κώδικα, χρησιμοποιήσαμε τον πίνακα στον οποίο αποθηκεύαμε τις τετράδες μας για ευκολία στην υλοποίηση. Θα μπορούσαμε να διαβάζουμε από το αρχείο που δημιουργούσαμε αλλά θεωρήσαμε πως μπορεί με αυτόν το τρόπο να υλοποιηθεί ευκολότερα.

Αυτό που σκεφτήκαμε ήταν πως αφού τα scores διαγράφονται με το πέρας του κάθε block συνάρτησης θα χρειαστεί να παράγουμε τον τελικό κώδικα κάθε φορά που η `get_token()`, στην δεύτερη προσπέλαση της, βρίσκει τις λέξεις για τέλος ενός block.

Για αυτόν τον λόγο όποτε τελειώνει ένα block ξεκινούσαμε από την τετράδα που είχαμε μείνει και μέχρι το τέλος των τετράδων παρήγαμε τον τελικό κώδικα.

Συγκεκριμένα ακολουθήσαμε τις οδηγίες από το Ηλεκτρονικό Σύγγραμμά για την λήψη των operators από τις τετράδες, δηλαδή αν για παράδειγμα λαμβάναμε την τετράδα `<,a,b,28`, αφού καλούσαμε την `loadnr(a,"t1"),(b,"t2")` η οποία θα παρήγαγε κάποιο κομμάτι του τελικού μετά θα προσθέταμε στο αρχείο `.s` κάποιες γραμμές της μορφής `"blt t1,t2,28"`.

Και με αυτόν τον τρόπο κυλούσε η παραγωγή του τελικού κώδικα, κάτι τελευταίο άξιο αναφοράς είναι η χρήση του πίνακα συμβόλων. Συγκεκριμένα οι συναρτήσεις για `load store` κλπ τον χρησιμοποιούσαν ώστε να γίνεται η αποθήκευση και η φόρτωση των σωστών μεταβλητών και τιμών από τις προηγούμενες στοίβες αλλά και της παρούσας στοίβας στους

temporary registers που χρησιμοποιούνται για την λειτουργία της γλώσσας risc-v assembly.

Ακολουθούν κομμάτια κώδικα για την επίδειξη των προαναφερθέντων.

```
1851 ~ def produce_final_code():
1852     global quads
1853     global count_Q
1854     global asm_code
1855     mkfile = open("final.asm","w")
1856     addi_exists = 0
1857     par_count = 0
1858     leng = len(quads) - count_Q
1859 ~     for i in range(leng):
1860         myQ = quads[count_Q]
1861 ~         if myQ.op == "begin_block" and myQ.op1 == pr_name:
1862             asm_code.append("LMain:\n")
1863             asm_code.append("    addi sp,sp," + str(offsets[curr_scope])+"\n")
1864             asm_code.append("    move gp,sp\n")
1865 ~         elif myQ.op == "begin_block":
1866             asm_code.append("L_" + str(myQ.op1) + ":\n")
1867             asm_code.append("    sw ra,(sp)\n")
1868 ~         else:
1869             asm_code.append("L_" + str(myQ.label)+"\n")
1870
1871
1872
1873 ~         if myQ.op in ops:
1874             produce_op(myQ)
1875 ~         elif myQ.op in jumps:
1876             produce_jump(myQ)
1877 ~         elif myQ.op == "halt":
1878             asm_code.append("    li a0,0\n")
1879             asm_code.append("    li a7,93\n")
1880             asm_code.append("    ecall\n")
1881 ~         elif myQ.op == "inp":
1882             produce_read(myQ)
1883 ~         elif myQ.op == "outp":
1884             produce_write(myQ)
1885 ~         elif myQ.op == "par" or myQ.op == "call" or myQ.op == "RET":
1886 ~             if not addi_exists:
1887                 ind = count_Q
1888                 breaker = 0
```

```

1886         if not addi_exists:
1887             ind = count_Q
1888             breaker = 0
1889             while True:
1890                 if quads[ind].op3 in funclist:
1891                     fl = 0
1892                     for j in scopes[curr_scope].ent_list:
1893                         if quads[ind].op3 == j.name:
1894                             fl = j.framelength
1895                             breaker = 1
1896                             break
1897                     asm_code.append("    addi fp,sp," + str(fl) + "\n")
1898                     addi_exists = 1
1899             if breaker:
1900                 break
1901             ind += 1
1902
1903     if myQ.op == "par":
1904         if myQ.op2 == "CV":
1905             par_count += 1
1906             loadvr(myQ.op1, "t1")
1907             off_par = 12 + (par_count - 1) * 4
1908             asm_code.append("    sw t1,-" + str(off_par) + "(fp)\n")
1909         if myQ.op2 == "REF":
1910             par_count += 1
1911             off_par = 12 + (par_count - 1) * 4
1912             my_par = None
1913
1914             for j in scopes[curr_scope].ent_list:
1915                 if myQ.op1 == j.name:
1916                     my_par = j
1917
1918             if my_par is not None:
1919
1920                 if my_par.ent_type == 2 and my_par.mode == "REF":
1921                     asm_code.append("    lw t0,-" + str(my_par.offset) + "(sp)\n")
1922                     asm_code.append("    sw t0,-" + str(off_par) + "(fp)\n")
1923
1924             else:

```

```

1924             else:
1925                 asm_code.append("    addi t0,sp,-" + str(my_par.offset) + "\n")
1926                 asm_code.append("    sw t0,-" + str(off_par) + "(fp)\n")
1927         else:
1928             search_sc = curr_scope - 1
1929             while search_sc >= 0:
1930                 for j in scopes[search_sc].ent_list:
1931                     if j.name == myQ.op1:
1932                         my_par = j
1933                     search_sc -= 1
1934
1935             if my_par.ent_type == 2 and my_par.mode == "REF":
1936                 gnlvcode(my_par.name)
1937                 asm_code.append("    lw t0,(t0)\n")
1938                 asm_code.append("    sw t0,-" + str(off_par) + "(fp)\n")
1939
1940             else:
1941                 gnlvcode(my_par.name)
1942                 asm_code.append("    sw t0,-" + str(off_par) + "(fp)\n")
1943
1944     elif myQ.op == "RET":
1945         my_offset = 0
1946         for j in scopes[curr_scope].ent_list:
1947             if myQ.op1 == j.name:
1948                 my_offset = j.offset
1949             break
1950         asm_code.append("    addi t0,sp,-" + str(my_offset) + "\n")
1951         asm_code.append("    sw t0,-8(fp)\n")
1952
1953     elif myQ.op == "call":
1954         is_parent = 0
1955         for j in scopes[curr_scope].ent_list:
1956             if j.name == myQ.op3:
1957                 is_parent = 1
1958         if is_parent:
1959             asm_code.append("    sw sp,4(fp)\n")
1960         else:
1961             asm_code.append("    lw t0,4(sp)\n")
1962             asm_code.append("    sw t0,4(fp)\n")

```

```

1803 def produce_op(myQ):
1804     if myQ.op == ":=":
1805         loadvr(myQ.op1,"t1")
1806         storerv("t1",myQ.op3)
1807     else:
1808         loadvr(myQ.op1,"t1")
1809         loadvr(myQ.op2,"t2")
1810
1811         if myQ.op == "+":
1812             asm_code.append("    add t1,t2,t1\n")
1813         elif myQ.op == "-":
1814             asm_code.append("    sub t1,t2,t1\n")
1815         elif myQ.op == "*":
1816             asm_code.append("    mul t1,t2,t1\n")
1817         elif myQ.op == "/":
1818             asm_code.append("    div t1,t2,t1\n")
1819
1820         storerv("t1",myQ.op3)
1821
1822 def produce_jump(myQ):
1823     if myQ.op == "jump":
1824         asm_code.append("    j L_" + str(myQ.op3) + "\n")
1825     else:
1826         loadvr(myQ.op1,"t1")
1827         loadvr(myQ.op2,"t2")
1828         if myQ.op == "=":
1829             asm_code.append("    beq t1,t2,L_" + str(myQ.op3)+"\n")
1830         elif myQ.op == "<":
1831             asm_code.append("    blt t1,t2,L_" + str(myQ.op3)+"\n")
1832         elif myQ.op == ">":
1833             asm_code.append("    bgt t1,t2,L_" + str(myQ.op3)+"\n")
1834         elif myQ.op == "<=":
1835             asm_code.append("    ble t1,t2,L_" + str(myQ.op3)+"\n")
1836         elif myQ.op == ">=":
1837             asm_code.append("    bge t1,t2,L_" + str(myQ.op3)+"\n")
1838         elif myQ.op == "<>":
1839             asm_code.append("    bne t1,t2,L_" + str(myQ.op3)+"\n")

```

```

1706 def loadvr(v,r):
1707     global curr_scope
1708     global scopes
1709     global asm_code
1710
1711     if v.isnumeric():
1712         asm_code.append("    li " + r + "," + v + "\n" )
1713     else:
1714         type_temp = 0
1715         my_of = 0
1716         for i in scopes[curr_scope].ent_list:
1717             if i.name == v:
1718                 if i.ent_type == 0 or i.ent_type == 3:
1719                     type_temp = 1
1720                     my_of = i.offset
1721                 if i.ent_type == 2:
1722                     if i.mode == "CV":
1723                         my_of = i.offset
1724                         type_temp = 1
1725                     elif i.mode == "REF":
1726                         my_of = i.offset
1727                         type_temp = 2
1728
1729
1730         if type_temp == 0:
1731             t = curr_scope - 1
1732             while t >= 0:
1733                 for i in scopes[t].ent_list:
1734                     if i.name == v:
1735                         if i.ent_type == 2:
1736                             if i.mode == "CV":
1737                                 type_temp = 3
1738                             else:
1739                                 type_temp = 4
1740                         elif i.ent_type == 1:
1741                             type_temp = 3
1742                 t-=1
1743         if type_temp == 1:
1744             asm_code.append("    lw "+r+"," +str(my_of)+"(sp)\n" )
1745         if type_temp == 2:
1746             asm_code.append("    lw t0,-" +str(my_of)+"(sp)\n" )
1747             asm_code.append("    lw " + r + ",(t0)\n")
1748         if type_temp == 3:
1749             gnlvcode(v)
1750             asm_code.append("    lw "+r+"," +str(my_of)+"(t0)\n")
1751         if type_temp == 4:
1752             gnlvcode(v)
1753             asm_code.append("    lw t0,(t0)")
1754             asm_code.append("    lw " + r + ",(t0)")
1755

```

Εν κατακλείδι, θα θέλαμε να πούμε πως πράγματι μας φάνηκε αρκετά χρήσιμη και ευχάριστη η υλοποίηση της προγραμματιστικής άσκησης.

Κύριες πηγές μας για την υποστήριξη και κατανόηση της άσκησης και της υλοποίησης της, ήταν οι διαφάνειες του μαθήματος και το Ηλεκτρονικό Σύγγραμμά.

Για βασικά πράγματα υλοποίησης της χρήσης της γλώσσας και γενικότερα του προγραμματιστικού κομματιού χρησιμοποιήσαμε searches στο [geeksforgeeks](#) και threads στο [StackOverflow](#)

Για debugging , εκτός από το δικό μας testing , χρησιμοποιήσαμε και τον debugger του [vscode](#).